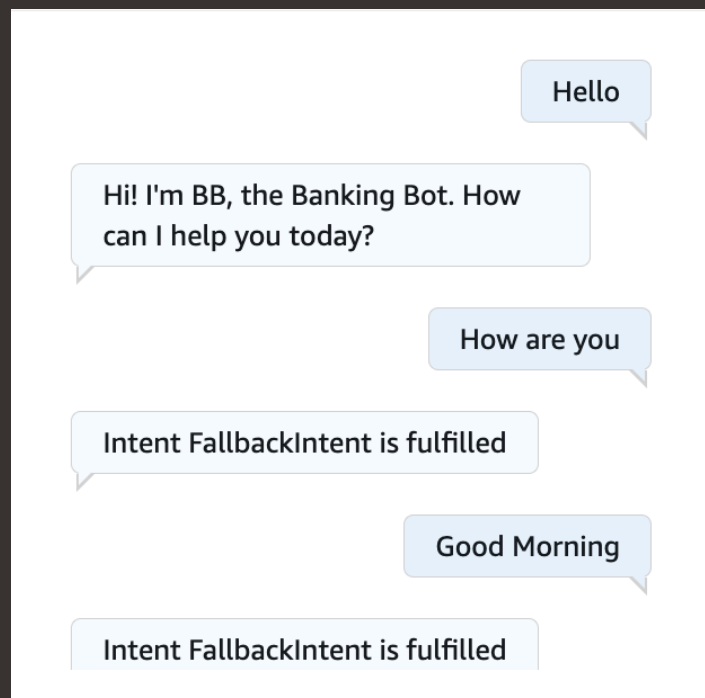




Build a Chatbot with Amazon Lex

CH Arpita Chowdhury





Introducing Today's Project!

Project overview

In this project, I will demonstrate how to build an intelligent chatbot using Amazon Lex that can understand natural language and interact with users through text or voice. I'm doing this project to learn the fundamentals of conversational AI, including intents, utterances, and bot configuration, and to understand how chatbots can be used in real-world applications like customer support and banking services.

What is Amazon Lex?

Amazon Lex is a fully managed AWS service for building conversational interfaces using voice and text, allowing developers to create chatbots that understand natural language, recognize user intent, and respond intelligently.

Key tools and concepts

Services I used were Amazon Lex and AWS IAM. Key concepts I learnt include how conversational AI works, defining intents and utterances, handling unknown inputs with FallbackIntent, using confidence scores to control intent matching, adding response variations and initial responses to improve user experience, and designing chatbots that feel more natural and user-friendly.



Personal reflections

One thing I didn't expect in this project was how much small design choices like fallback messages, confidence scores, and response variations affect how natural and user-friendly a chatbot feels, even when the underlying functionality is simple.

Project timeline

This project took me approximately 2 hours. The most challenging part was understanding how intents, fallback behavior, and confidence scores work together to control how the chatbot responds. It was most rewarding to see BankerBot respond naturally to user input and handle unknown phrases gracefully, making the chatbot feel polished and interactive.



Setting up a Lex chatbot

Approach to chatbot setup

In this step, I will set up my Lex chatbot because it allows me to define how the bot interacts with users, understands their input, and responds to conversations using natural language.

Chatbot creation process

I created my chatbot from scratch with Amazon Lex. Setting it up took me about 15 minutes, as I configured the bot's basic settings, language, and initial behavior before moving on to intents and responses.

IAM role configuration

While creating my chatbot, I also created a role with basic permissions because Amazon Lex needs permission to access and interact with other AWS services (like logging to CloudWatch and invoking integrations) so the chatbot can run securely and function correctly without granting unnecessary access.



Intent classification confidence score

In terms of the intent classification confidence score, I kept the default value of 0.40, which means the chatbot must be at least 40% confident that it understands the user's intent before responding; otherwise, it will trigger the fallback behavior to avoid incorrect or confusing replies.

The screenshot shows a configuration panel for a chatbot. At the top, it says 'Language: English (US)'. Below this is a 'Select language' dropdown menu with 'English (US)' selected. There is a 'Description - optional' text area with a note 'Maximum 2000 characters.' and an 'Enable speech-to-speech inference' checkbox which is unchecked. The 'Voice interaction' section includes a dropdown for 'The text-to-speech voice that your bot uses to interact with users.' with 'Danielle' selected, and a 'Voice sample' text input with the text 'Hello, my name is Danielle. Let me know how I can assist you.' and a 'Play' button. The 'Speech model preference' section has a dropdown for 'The preferred speech model to use for speech-to-text' with 'None' selected. Finally, the 'Intent classification confidence score threshold' is set to '0.40' in a text input, with a note 'Min: 0.00, max: 1.00.'



Intents

In this step, I will create and train a greeting intent for BankerBot because this teaches the chatbot how to recognize common greetings and respond appropriately, allowing it to start conversations naturally and interact with users in a friendly, conversational way.

Creating my first intent

Intents are the goals or purposes behind what a user says to the chatbot, helping Amazon Lex understand what the user wants to do and determine how the bot should respond.

I created my first intent, `WelcomeIntent`, to recognize when a user greets the chatbot and respond with a friendly welcome message, allowing BankerBot to start conversations naturally and set the tone for further interactions.

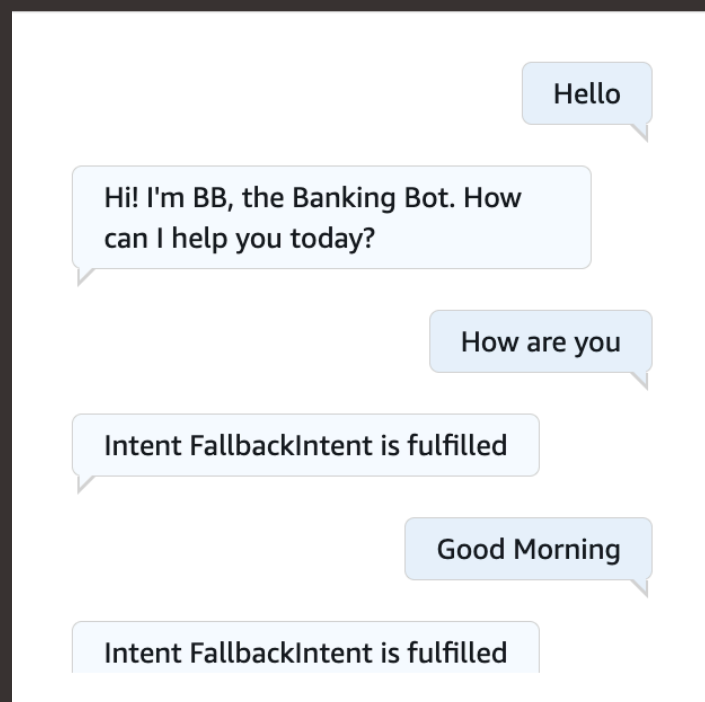
Testing chatbot responses

I launched and tested my chatbot, which could respond successfully when I entered greetings like “Hello,” “Hiya,” or “Help me.” These inputs correctly triggered my `WelcomeIntent`, and the bot responded with its greeting message, confirming that the intent training and basic conversation flow were working as expected.



Handling unrecognized inputs

My chatbot returned the error message “Intent FallbackIntent is fulfilled” when I entered phrases like “How are you” or “Good morning.” This error message occurred because those inputs did not match any of the defined sample utterances in my WelcomeIntent, so Amazon Lex could not confidently map them to an existing intent and instead routed the request to the FallbackIntent, which is triggered whenever the bot doesn’t understand the user’s input.





Configuring FallbackIntent

In this step, I will customize the FallbackIntent to return clear, user-friendly responses because the default message is confusing for users. By adding helpful fallback messages, the chatbot can politely let users know it didn't understand their input and guide them on what to say next, making the conversation feel more natural and easier to use.

When FallbackIntent triggers

FallbackIntent is a default intent in every chatbot that gets triggered when the user's input does not match any of the defined intents or when the chatbot is not confident enough to understand what the user is asking.

I wanted to configure FallbackIntent because, by default, it returns a technical message that isn't user-friendly, and I wanted my chatbot to respond in a clearer, more helpful way when it doesn't understand a user's input.

My FallbackIntent configuration

To configure FallbackIntent, I edited its response messages and added friendly, helpful replies so the chatbot can clearly let users know it didn't understand their input and guide them on what to say next instead of showing a confusing default message.

I also added variations, which means for an end user the chatbot won't repeat the same fallback message every time. Instead, it will respond with different helpful messages like Hmm could you try rephrasing that? I can help you find your account balance, transfer funds and make a payment.



FallbackIntent with Variations

wasssup

Sorry I am having trouble understanding. Can you describe what you'd like to do in a few words? I can help you find your account balance, transfer funds and make a payment.



Initial Responses

In this project extension, I'm about to enhance BankerBot's FallbackIntent by adding engaging initial responses with multiple variations. I'm doing this so that when the chatbot doesn't understand a user's input, it can respond in a more friendly, natural, and conversational way—improving the overall user experience and making the bot feel more polished and realistic.

Setting up initial responses

Initial responses are the messages a chatbot sends immediately while it's processing a user's input, helping keep the conversation engaging instead of feeling silent or unresponsive. I've set up initial responses for FallbackIntent, using variations like "Hmmm, this is interesting...", "One moment...", and "Oh no! Wait please..." so users know the bot is working and the interaction feels more natural and human-like.

Initial response implementation

The initial response messages I set up are short, friendly phrases like "Hmmm, this is interesting...", "One moment...", and "Oh no! Wait please...". For the user, this means the chatbot feels more responsive and conversational, reassuring them that their input is being processed even when the bot needs extra time to respond.



Arpita Chowdhury

NextWork Student

nextwork.org

my name is

Oh no! Wait please....

Sorry I am having trouble understanding. Can you describe what you'd like to do in a few words? I can help you find your account balance, transfer funds and make a payment.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

